Imitation-Learning Data Collection for MABEL: A Single-Tap Full-Bus Recorder from rosbag2/MCAP to Rerun

MABEL project — `server/`

October 17, 2026

Abstract

This document describes the MABEL data-collection subsystem: the pipeline that captures complete teleoperation demonstrations — robot proprioception, the human operator's commands, every camera and depth stream, point clouds, the volumetric environment mesh, the 2-D occupancy map, odometry, and the full transform tree — into a single self-describing dataset suitable for learning *long-horizon navigation and manipulation* policies. A demonstration is started and stopped with one tap in the iOS app; the teleoperation bridge that owns the simulated robot spawns a `ros2 bag record` child that writes the entire ROS 2 message bus to a **rosbag2 / MCAP** container on the local host (never pushed to version control). A companion converter replays a bag into a **Rerun** recording, reconstructing the `tf` tree as a spatial entity hierarchy so the whole episode — images, depth, clouds, mesh, map, paths, and every joint/command time series — is visualized in one coherent, time-synchronized scene. We document the trigger path, the storage-format rationale, the recorded topic schema, the recorder lifecycle, and the bag-to-Rerun mapping, and we report the current verification state.

Keywords: imitation learning, demonstration recording, rosbag2, MCAP, Rerun, ROS 2, teleoperation, mobile manipulation, navigation, MuJoCo.

1 Overview and motivation

Learning long-horizon behaviours that interleave *navigation* (drive to a place) and *manipulation* (act on objects there) requires demonstrations that are simultaneously rich and *temporally coherent*: the policy must see the operator's commands, the robot's realized motion, and the perceptual context (what the cameras, depth sensors, map, and reconstructed mesh showed at that instant) on one clock. The MABEL teleoperation stack already produces all of these as ROS 2 topics when the simulated (or real) robot runs under `sim_ros_server.sh`; the data-collection subsystem's job is to capture them, losslessly and in step, on operator demand.

Two design commitments shape the implementation:

1. **Record the bus, not a bespoke episode format.** Everything the policy could want is already published on the ROS graph with correct headers and timestamps. The recorder therefore writes the *wire* directly to a standard container, rather than re-serializing a hand-picked struct. This captures message types whose Python bindings are not even importable in the recording process (for example the volumetric mesh), and it inherits the entire ROS introspection, replay, and conversion ecosystem for free.
2. **Separate the storage truth from the viewer.** The dataset of record is the MCAP bag. Visualization is a derived artifact produced by a converter that can be re-run at any time

with a different layout or selection, so the raw data is never perturbed by a desire to look at it differently.

2 Dataflow

A single button in the app's Cockpit drives both the existing per-episode recorder and the new full-bus rosbag. The control message rides the same WebSocket envelope as all other operator intent (see the teleoperation-protocol document); the bridge, which owns the MuJoCo physics and the ROS publishers, queues the request onto its simulation thread, which manages a recorder child process.

```
iOS app  (Cockpit "Start Collection")
   |  RobotController.startDataCollection(label)
   |  -> RobotCommand.dataCollect(active, label)
   v  WebSocket  {"type":"data_collect","payload":{"active":true,"label":...}}
sim_teleop_bridge.py
   |  WireMessageType.DATA_COLLECT handler  ->  _data_collect_pending queue
   |  (net thread only enqueues; the sim thread owns the recorder subprocess)
   v
BagRecorder.start(label)                          [server/data_collection.py]
   |  spawn:  ros2 bag record --storage mcap --storage-preset-profile zstd_fast
   v
recordings/<YYYYmmdd_HHMMSS>_<label>/             [local host; git-ignored]
   |    +-- <bag>_0.mcap          (the messages)
   |    +-- metadata.yaml         (rosbag2 index)
   |    +-- session.json          (label, git commit, topic list, timing)
   v
bag_to_rerun.py  <bag>            [run in the 'mabel' conda env]
   |  rosbag2_py SequentialReader -> deserialize -> rerun log
   v
<bag>/rerun.rrd  ->  rerun <bag>/rerun.rrd        (3-D scene + cameras + series)
```

The bridge stops the child with `SIGINT` so rosbag2 flushes its chunks and writes `metadata.yaml`; an `atexit` hook stops any live recording on server shutdown so a bag is never left unfinalized. Recording start/stop is idempotent, and a recording survives the operator's client disconnecting (it is owned by the server, not the session).

3 Storage format: rosbag2 / MCAP

The dataset container is **MCAP**, the default rosbag2 storage plugin in this workspace (ROS 2 Jazzy, RoboStack/conda). MCAP was chosen because it is:

- **Self-describing.** Each bag embeds the message schemas and per-topic QoS, so it replays and introspects with no external `.msg` files — a property that matters for a dataset meant to outlive the exact source tree that produced it.
- **Complete and ordered.** Every message on the selected topics is stored with both its header stamp and its receive time, preserving the temporal alignment the policy needs.
- **Efficient.** The recorder enables the `zstd_fast` chunk-compression preset, which typically shrinks image-heavy bags several-fold at negligible CPU cost.

- **Standard and tooled.** `ros2 bag info/play`, the MCAP CLI, and a growing set of third-party loaders (including Rerun's own) read it directly.

Bags are written under `<repo>/recordings/` by default (override with the `MABEL_RECORDINGS_DIR` environment variable). This tree is **git-ignored** (`/recordings/`, `*.mcap`, `*.rrd`); collected data is local-only and is never committed.

4 Recorded schema

The default capture is a curated set of **52 topics**, declared as labelled groups in `data_collection.py`. The grouping makes explicit what each block contributes to a navigation+manipulation dataset and makes trimming obvious. Topics that are not currently being published (for example the volumetric mesh when SLAM is not running) are simply never written — rosbag2 records a named topic only if and when it appears.

Transforms and clock. Without the transform tree the dataset is spatially incoherent, so these are always on.

```
/tf            /tf_static            /clock
```

Proprioception (measured state). The robot configuration out of the controller, plus odometry and the front inertial unit.

```
/joint_states           /arms/joint_states      /body/joint_states
/left_hand/joint_states /right_hand/joint_states
/odom                   /odom_ground_truth      /camera/front/imu
```

Operator commands (the supervision signal). The setpoints *into* the controller — the targets a policy is trained to reproduce.

```
/mabel_cmd          /arms/command       /body/command
/left_hand/command  /right_hand/command /goal_pose          /cmd_vel
```

Control-mode status. Which teleoperation regime was active, and whether the body was tucked ready for navigation.

```
/mabel/control_mode      /mabel/body_ready
```

Primary perception (head + front base RGB-D, plus SLAM stereo). Colour, depth, and a colourized point cloud for the front camera and the head, the infrared stereo pair the visual-SLAM front-end consumes, and every matching `camera_info` so depth back-projects metrically.

```
/camera/front/color/image_raw        (+ /camera/front/color/camera_info)
/camera/front/depth/image_rect_raw   (+ /camera/front/depth/camera_info)
```

```
/camera/front/depth/points
/camera/front/infra1/image_rect_raw  (+ .../infra1/camera_info)
/camera/front/infra2/image_rect_raw  (+ .../infra2/camera_info)
/camera/zed/left/image_raw           (+ /camera/zed/left/camera_info)
/camera/zed/right/image_raw          (+ /camera/zed/right/camera_info)
/camera/zed/depth/image_rect_raw     (+ /camera/zed/depth/camera_info)
/camera/zed/depth/points
```

Auxiliary views (manipulation eyes + situational awareness).

```
/camera/left_wrist/image_raw   (+ camera_info)
/camera/right_wrist/image_raw  (+ camera_info)
/camera/body/image_raw         (+ camera_info)
/camera/back/color/image_raw   (+ camera_info)
/camera/back/depth/image_rect_raw (+ camera_info)
```

Mapping, mesh, and planning. The LiDAR scan, the 2-D occupancy grid and its inflated cost field, the volumetric environment mesh (as both a colourized cloud and a triangle list), and the planner's path.

```
/scan   /map   /map_costmap
/nvblox_node/mesh_pointcloud   /nvblox_node/mesh_marker   /plan
```

Two environment variables override the default at session start: `MABEL_BAG_ALL=1` records every topic on the graph (`ros2 bag record -a`); `MABEL_BAG_TOPICS="/a /b ..."` replaces the list entirely.

5 App-recorded episodes: the operator's control state per frame

Episodes recorded from the apps (*Record*) are written by `curation_recorder.py` in the Data Curation studio's H5 format (`obs/cam/*`, `obs/joint_pos`, ..., `action/ctrl`). Since 2026-10-16 every frame also records *which parts of the operator's body were driving the robot* — the ablation switches (neck / torso / upper vs whole body) can change mid-episode, so this is per frame, never only per episode:

key	dtype	meaning
`obs/control/tracking`	int8[5]	neck, arms, hands, torso, lift: 1 driven, 0 frozen at home, −1 unknown
`obs/control/scope`	int8	0 local (upper body), 1 global (whole body), −1 unknown
`obs/control/method`	int8	0 navigation, 1 joint, 2 wrist, 3 retarget, −1 other
`obs/control/flags`	int8[3]	gravity comp, whole-body arm profile, e-stop
`obs/control/engaged`	int8	1 = retargeter engaged (robot following the operator)

The legend rides each file's `schema_json` as `control_encoding` (numeric codes: h5py cannot store numpy unicode arrays). Two defects were fixed on the way: the capture sample already carried the control spec but `_write_frame` silently dropped it, so no earlier episode has these labels; and the spec was read on the worker thread at render time, which could straddle a toggle — it is now snapshotted in `RecordPump.maybe_capture` under the same lock as the physics copy.

Verified. `server/tests/test_control_recording.py` (5 tests, including a mid-episode toggle written through the real IL `EpisodeRecorder` and read back as `[1,1,1,0,0,0]`); and an end-to-end run of the gateway's own capture path (MuJoCo model → `RecordPump` with camera renders → `CurationRecorder` → H5) over 8 s with three switches (neck frozen at 2 s; upper body + torso frozen at 4 s; whole body at 6 s): 53 frames, **0** mislabelled away from a switch boundary.

6 Recorder lifecycle

`BagRecorder` (in `data_collection.py`) owns at most one live `ros2 bag record` child and is driven entirely from the bridge's simulation thread, so it needs no internal locking. Its responsibilities are:

1. **Naming.** Construct `recordings/<timestamp>_<slug>/`, where the slug is a filesystem-safe form of the operator's label.
2. **Launch.** Spawn `ros2 bag record` in a new process group (`start_new_session=True`) with the MCAP storage and `zstd_fast` preset and either the curated list or `-a`. Crucially the bag directory is *not* pre-created: rosbag2 owns and creates it, and errors if it already exists.
3. **Early-failure detection.** Briefly poll the child after launch; if it exits immediately (bad topic spec, missing plugin, unwritable directory) the error is surfaced to the caller instead of silently producing no data.
4. **Metadata.** Write a `session.json` sidecar with the label, ISO start time, the resolved topic list, the `ROS_DISTRO`, the git commit, the exact command, and the host; on stop, fold it into the finished bag directory with the end time and duration.
5. **Clean stop.** Signal the process group with `SIGINT` (so rosbag2 finalizes the MCAP and `metadata.yaml`), wait with a timeout, and escalate to `SIGTERM` only if it overruns.
6. **Status.** Expose a small dictionary (active, label, directory, elapsed seconds, session count, last error) that the bridge folds into its robot-state broadcast, and that detects a child which died on its own.

The recorder is also runnable standalone for testing without the app or bridge: `python3 server/data_collection.py <label>` records until interrupted.

7 Conversion to Rerun

`bag_to_rerun.py` replays a bag into a Rerun recording. It must run in the `mabel` conda environment, which uniquely has `rosbag2_py`, the ROS message typesupport, *and* the Rerun SDK together. It opens the MCAP with a `SequentialReader`, deserializes each message with the type advertised in the bag, and logs it to a `.rrd` (or a live viewer with `--spawn`). Message types whose Python bindings are absent (for example `nvblox_msgs/Mesh`) are skipped with a count; the same geometry still arrives through the mesh's marker and point-cloud topics.

Spatial coherence via the transform tree. Rerun composes transforms down the entity-path hierarchy, and `tf` provides each child frame's pose in its parent. The converter therefore maintains the child→parent map from `/tf` and `/tf_static` and logs every frame's transform at the entity path `world/.../<parent>/<child>`, reproducing the kinematic tree exactly. It reads the bag in **two passes** — first resolving the complete tree, then logging — so every entity is placed at its final path; a single-pass converter would log a frame seen before its full parent chain (e.g. the latched `/tf_static` at $t = 0$, before `odom`→`base_link`) at a shorter path that never matches the final one, stranding that whole subtree (and the robot meshes hung from it) at the origin. Static transforms are logged once, time-independently. All sensor data is then logged *under its own frame's path*, so

images, depth, clouds, scans, mesh, map cells, and paths all appear in their correct place in a single 3-D view. The world root is declared right-handed Z-up to match the ROS convention.

Robot-body overlay. The converter also draws the robot itself: it parses the `mabel_description` URDF and logs each link's visual mesh, transformed by its visual origin into the link frame, *statically* at that link's `tf` entity path. Because the transforms place every link each frame, the body articulates live with the recorded motion. This is enabled by default (`--no-robot` disables it, `--urdf` selects a different model). Camera frustums are drawn at a fixed image-plane distance (`--frustum`) so they do not visibly rescale as the scene's bounding box changes. Point clouds and laser scans are drawn with *screen-space* (UI-point) radii (`--point-size`) rather than a world-space radius, so they stay visible at any zoom — a fixed world radius such as 1 cm falls below one pixel when the view fits the whole scene and the cloud appears to vanish. Depth clouds are also range-clipped (`--max-range`, default 8 m): a depth camera emits a few dozen far-range fliers per frame (out to ∼18 m here, while 95 % of points sit inside ∼4 m), and because that flier *count* jitters frame-to-frame the scene's bounding box pulses and the 3-D view's auto-camera continually refits — which reads as the cloud shaking and disappearing while scrubbing. Clipping the fliers fixes the box. Finally, each frame's first transform is also logged *statically* as a fallback, so the robot is fully posed at the very first timeline instant; otherwise a dynamic frame has no pose until its first `/tf` message and snaps in from the origin, which reads as an initial "shake" on playback.

Table 1: ROS message → Rerun archetype mapping.

ROS message	Rerun archetype	Entity path
`tf` / `tf_static`	`Transform3D`	`world/.../<frame>` (tree)
`Image` (colour)	`Image`	`<cam frame>/image`
`Image` (depth)	`DepthImage`	`<cam frame>/depth`
`CameraInfo`	`Pinhole`	`<cam frame>` (static)
`PointCloud2`	`Points3D`	`<frame>/points`
`Marker` (mesh, triangles)	`Mesh3D`	`<frame>/nvblox_mesh`
`OccupancyGrid`	`Image` + `Points3D`	`maps2d/<name>`, `<map>/<name>`
`LaserScan`	`Points3D`	`<frame>/scan`
`Path`	`LineStrips3D`	`<frame>/plan`
`PoseStamped` (goal)	`Transform3D`	`<frame>/goal`
`JointState` (state & command)	`Scalars`	`joints/<group>/<joint>`
`Imu` / `Odometry` / `Twist` / `MabelSwerveCmd`	`Scalars`	`imu/...`, `odom/...`, `cmd/...`
`String` (control mode)	`TextLog`	`mabel_control_mode`

Per-message time uses the message header stamp when present and the bag receive time otherwise, placed on a single `ros_time` timeline so every stream scrubs together. The converter emits a default *blueprint*: a 3-D scene beside a tabbed set of 2-D camera views, the joint/command/IMU time series, and the 2-D map. Two practical encoding details proved necessary: occupancy values are widened to 16-bit before the grayscale remap (8-bit arithmetic overflows under NumPy's strict integer promotion), and joint names containing spaces (e.g. `Left Arm 1`) are sanitized because Rerun entity paths forbid unescaped whitespace.

8 Reproducibility and environment

The recorder runs inside the bridge process, which already sources ROS 2 Jazzy from the `mabel` conda environment (RoboStack); the `ros2` CLI and the MCAP storage plugin are therefore on

the path. The converter additionally needs the Rerun SDK (`rerun-sdk==0.26.2`), installed into the same environment so that bag deserialization and Rerun logging share one interpreter. The `session.json` written with every bag records the git commit and the exact recording command, so a dataset can be traced back to the source tree that produced it.

9 Verification status

The end-to-end path — a publisher emitting representative messages (`tf`, joint states, a depth image with its `camera_info`, a point cloud, a LiDAR scan, a latched occupancy grid, and a control-mode string), `BagRecorder` writing an MCAP, and `bag_to_rerun.py` converting it — has been exercised and produces a clean bag (correct `metadata.yaml`, folded `session.json`, latched topics captured) and a loadable `.rrd` with no skipped messages or warnings. Three defects found during that exercise (a pre-created bag directory rejected by rosbag2, the 8-bit occupancy overflow, and whitespace in joint entity paths) were fixed.

What remains unverified: a capture against a live `sim_ros_server.sh` session with the SLAM stack running (real camera, mesh, and map data rates — hence realistic bag size and conversion time), and a compile of the iOS additions (the Swift edits follow the existing command patterns but were not built on this Linux host).

10 Future work

Natural extensions are a dataset index that aggregates `session.json` across recordings, automatic depth-to-cloud back-projection from `camera_info` for cameras whose clouds are not recorded, and a selectable Rerun blueprint per task type. The robot-body overlay (§7) could additionally colour links by kinematic group or load a decimated mesh set for faster loading of large recordings.